PARA QUEM JÁ SABE HTML E CSS · 10 MÓDULOS

JavaScript do Zero

```
console
.log(
"olá"
);
```

Variáveis, decisões, funções, listas, o DOM e eventos — para dar vida a páginas web, com exemplos, exercícios e um projeto final.

ROTEIRO

Dez módulos, passo a passo

01
Primeiros passos

02
Variáveis e tipos

03
Operadores e decisões

04
Funções

05
Arrays e ciclos

06
Objetos

07
O DOM

08
Eventos e formulários

09
Guardar e carregar dados

10
Depurar e rever

Final
Projeto: a biblioteca interativa

Anexo
Soluções

Estrutura, aspeto e comportamento

HTML

O que existe

Títulos, parágrafos, botões — os substantivos

```
<button>Reservar</button>
```

CSS

Como se vê

Cores, letras, layout — os adjetivos

```
button {
  background: gold;
}
```

JavaScript

O que acontece

Reagir a cliques, calcular, mudar a página — os verbos

```
botao.addEventListener(
  "click", reservar
);
```

Ligar um ficheiro .js

index.html

```html
<head>
  <link rel="stylesheet" href="estilos.css">
  <script src="app.js" defer></script>
</head>
```

app.js

```js
// O meu primeiro programa
console.log("Olá, Biblioteca do Bairro!");
```

> **defer**: o script só corre depois de o HTML estar todo carregado

Consola (F12)

```
Olá, Biblioteca do Bairro!                app.js:2
> |
```

A consola, o nosso laboratório

- Escreve-se uma instrução, Enter, e vê-se o resultado
- Os erros aparecem a vermelho, com a explicação
- Maiúsculas contam: **console** não é **Console**
- Comentários: **// até ao fim da linha**

Consola

```
> 2026 - 1987
← 39

> 1250 * 2
← 2500

> "Biblio" + "teca"
← 'Biblioteca'

> console.log("Bom dia!")
Bom dia!

> consola.log("Bom dia!")
Uncaught ReferenceError: consola is not defined
```

EXERCÍCIO 01 · 15 MIN

Olá, consola

01 Crie app.js e ligue-o à página inicial, com defer

02 Mostre na consola uma mensagem de boas-vindas

03 Na consola, calcule há quantos anos abriu a biblioteca (1987)

04 No topo do app.js, um comentário com o seu nome

05 Escreva console.lg de propósito e leia o erro

Concluído quando…

- A mensagem aparece na consola ao abrir a página
- Sabe dizer em que ficheiro e linha está o erro
- Depois de corrigido, a consola fica sem vermelho

Variáveis: caixas com nome

app.js

```
const nome = "Biblioteca do Bairro";
let livros = 1250;

livros = livros + 30;   // let pode mudar
console.log(nome);
console.log(livros);

nome = "Outra";         // const não muda!
```

const por omissão

let se o valor muda

var: antigo, evitar

Consola

```
Biblioteca do Bairro
1280
Uncaught TypeError: Assignment to constant variable.
```

Tipos de dados

Tipo	Exemplo	Serve para
string	"Os Maias"	Texto, entre aspas
number	1250 · 0.20	Números, com ponto decimal
boolean	true · false	Sim ou não
undefined	let x;	Ainda sem valor
null	null	Vazio de propósito
array	["a", "b"]	Listas (módulo 5)
object	{ ano: 1888 }	Fichas (módulo 6)

Consola

```
> typeof 1250
← 'number'

> typeof "1250"
← 'string'

> "5" + 2
← '52'  cuidado!

> Number("5") + 2
← 7
```

Trabalhar com texto

app.js

```
const titulo = "Os Maias";
const autor = "Eça de Queirós";
const ficha = `${titulo}, de ${autor}`;

console.log(ficha);
console.log(titulo.length);
console.log(titulo.toUpperCase());
console.log(autor.includes("Eça"));
```

Consola

```
Os Maias, de Eça de Queirós
8
OS MAIAS
true
```

Acentos graves `` e **${ }** metem variáveis no texto

Métodos com ponto: **texto.metodo()**

EXERCÍCIO 02 · 20 MIN

Os dados da biblioteca

01 Guarde em const o nome e a morada da biblioteca

02 Guarde em let os livros disponíveis: 1250

03 Registe 3 empréstimos: diminua a variável e mostre-a

04 Com uma template literal: «A Biblioteca do Bairro tem 1247 livros disponíveis.»

05 Mostre o typeof de cada variável e o nome em maiúsculas

Concluído quando…

- A frase aparece na consola com o número certo
- O número 1247 é calculado, não escrito à mão
- Só usa let onde o valor muda

Calcular, comparar, combinar

Aritméticos

```
1250 + 30 → 1280
10 / 4 → 2.5
10 % 4 → 2
2 ** 3 → 8
```

% é o resto da divisão

Comparação

```
22 > 15 → true
5 === 5 → true
5 === "5" → false
5 !== 6 → true
```

Usar **===**: compara valor e tipo

Lógicos

```
true && false → false
true || false → true
!true → false
```

&& e · **||** ou · **!** não

Decidir com if e else

app.js

```
const dias = 22;   // dias com o livro
const prazo = 15;

if (dias <= prazo) {
  console.log("Dentro do prazo");
} else if (dias <= prazo + 5) {
  console.log("Atrasado: devolva hoje");
} else {
  const multa = (dias - prazo) * 0.50;
  console.log(`Multa: ${multa.toFixed(2)} €`);
}
```

Consola, com dias diferentes

dias = 10

```
Dentro do prazo
```

dias = 18

```
Atrasado: devolva hoje
```

dias = 22

```
Multa: 3.50 €
```

> Só um dos blocos corre: o primeiro cuja condição é verdadeira

EXERCÍCIO 03 · 25 MIN

Aberta ou fechada?

01 Crie as variáveis dia (texto, ex.: "sábado") e hora (número)

02 De segunda a sexta, a biblioteca abre das 9 às 19 h

03 Ao sábado abre das 10 às 13 h; ao domingo está fechada

04 Mostre na consola «Aberta» ou «Fechada»

05 Teste com pelo menos quatro combinações de dia e hora

Concluído quando…

- Domingo dá sempre «Fechada»
- Às 19 h em ponto de quarta, já está fechada
- Usa ===, && e || em vez de ifs repetidos

Funções: receitas reutilizáveis

app.js

```
function calcularMulta(dias) {
  const prazo = 15;
  if (dias <= prazo) {
    return 0;
  }
  return (dias - prazo) * 0.50;
}

console.log(calcularMulta(10));
console.log(calcularMulta(22));
console.log(calcularMulta(30));
```

Consola

```
0
3.5
7.5
```

dias é o parâmetro: o valor de entrada

return devolve o resultado e termina a função

Define-se uma vez, chama-se quantas vezes quiser

Arrow functions e âmbito

app.js

```
// A mesma função, escrita de duas formas
function dobro(n) { return n * 2; }
const dobro2 = (n) => n * 2;

// Âmbito: cada variável vive no seu bloco
const biblioteca = "do Bairro";

function saudar() {
  const mensagem = "Bem-vindo à Biblioteca";
  console.log(mensagem, biblioteca);
}
saudar();
console.log(mensagem);  // fora do bloco
```

Consola

```
Bem-vindo à Biblioteca do Bairro
ReferenceError: mensagem is not defined
```

- **=>** é a forma curta, muito usada com arrays
- O que nasce dentro de { } não existe lá fora
- O que está fora vê-se lá dentro

EXERCÍCIO 04 · 30 MIN

Funções da biblioteca

01 calcularMulta(dias): 0,50 € por dia a partir do 16.º dia

02 estaAberta(dia, hora): a lógica do exercício 3, a devolver true ou false

03 formatarEuros(valor): devolve, por exemplo, «3,50 €»

04 mensagemMulta(dias): «Multa a pagar: 3,50 €» ou «Sem multa»

05 Teste cada função com três valores diferentes

Concluído quando…

- As funções usam return, sem console.log lá dentro
- estaAberta("domingo", 11) devolve false
- mensagemMulta usa as outras duas funções

Arrays: listas de valores

app.js

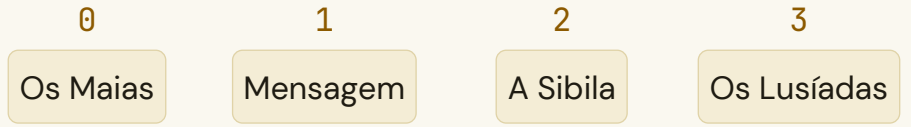
```
const livros = ["Os Maias", "Mensagem", "A Sibila"];

console.log(livros[0]);      // o primeiro
console.log(livros.length);

livros.push("Os Lusíadas");  // junta no fim
console.log(livros.length);
console.log(livros.includes("Mensagem"));
console.log(livros.indexOf("Mensagem"));
```

Consola

```
Os Maias
3
4
true
1
```



Ciclos: repetir sem copiar

app.js

```
const livros = ["Os Maias", "Mensagem", "A Sibila"];

// for...of: para cada livro da lista
for (const livro of livros) {
  console.log(`- ${livro}`);
}

// for clássico: com um contador
for (let i = 0; i < livros.length; i++) {
  console.log(`${i + 1}. ${livros[i]}`);
}
```

Consola

```
- Os Maias
- Mensagem
- A Sibila
```

```
1. Os Maias
2. Mensagem
3. A Sibila
```

for...of quando só interessa cada valor

for com contador quando a posição também importa

Métodos de arrays

```
const anos = [1888, 1934, 1954, 1995];  // anos de publicação
```

map

transforma cada um

```
anos.map(a => 2026 - a)
→ [138, 92, 72, 31]
```

filter

fica com os que passam

```
anos.filter(a => a > 1900)
→ [1934, 1954, 1995]
```

find

o primeiro que passa

```
anos.find(a => a > 1900)
→ 1934
```

forEach

faz algo com cada um

```
anos.forEach(a => console.log(a))
→ mostra os 4, um a um
```

EXERCÍCIO 05 · 25 MIN

Lista de leituras

01 Crie um array com cinco títulos de livros

02 Junte mais dois com push e mostre quantos são

03 Mostre a lista numerada: «1. Os Maias», «2. …»

04 Com filter, fique só com os títulos com mais de 10 letras

05 Com map, crie uma lista com os títulos em maiúsculas

Concluído quando…

- A numeração começa em 1, não em 0
- O array original fica igual depois de filter e map
- Nenhum título aparece escrito duas vezes no código

Objetos: fichas com propriedades

app.js

```
const livro = {
  titulo: "Os Maias",
  autor: "Eça de Queirós",
  ano: 1888,
  disponivel: true,
};

console.log(livro.titulo);
livro.disponivel = false;   // foi emprestado
livro.estante = "B3";      // nova propriedade
console.log(livro);
```

Consola

```
Os Maias
{titulo: 'Os Maias',
 autor: 'Eça de Queirós',
 ano: 1888,
 disponivel: false,
 estante: 'B3'}
```

- Pares **chave: valor**, separados por vírgulas
- O **ponto** lê, altera e acrescenta propriedades

Um catálogo: arrays de objetos

app.js

```
const catalogo = [
  { titulo: "Os Maias", ano: 1888, disponivel: true },
  { titulo: "Mensagem", ano: 1934, disponivel: false },
  { titulo: "A Sibila", ano: 1954, disponivel: true },
];

const livres = catalogo.filter(l => l.disponivel);
console.log(livres.map(l => l.titulo));

const achado = catalogo.find(l => l.ano === 1934);
console.log(achado.titulo);
```

Consola

```
['Os Maias', 'A Sibila']
Mensagem
```

> Cada livro é um objeto; o catálogo é o array

> **l** é cada livro, à vez

> É assim que chegam os dados de um servidor

EXERCÍCIO 06 · 30 MIN

O catálogo em objetos

01 Crie o array catalogo com seis livros: titulo, autor, ano e disponivel

02 Mostre «Título — Autor (ano)» para cada livro

03 Conte quantos livros estão disponíveis

04 Encontre e mostre o livro mais antigo

05 emprestar(titulo): marca como emprestado, ou avisa se não puder

Concluído quando...

- A contagem muda ao trocar um disponivel no array
- Emprestar um livro que não existe não dá erro
- Emprestar duas vezes o mesmo livro mostra um aviso

Encontrar elementos na página

index.html

```html
<header><h1>Biblioteca do Bairro</h1></header>
<main>
  <p class="aviso">Fechado no feriado</p>
  <ul id="lista-livros"></ul>
</main>
```

app.js

```js
const titulo = document.querySelector("h1");
const aviso = document.querySelector(".aviso");
const lista = document.querySelector("#lista-livros");
const paragrafos = document.querySelectorAll("p");
```

A árvore DOM

```
document
  body
    header
      h1
    main
      p.aviso
      ul#lista-livros
```

Os seletores são os mesmos do CSS

Alterar texto, classes e atributos

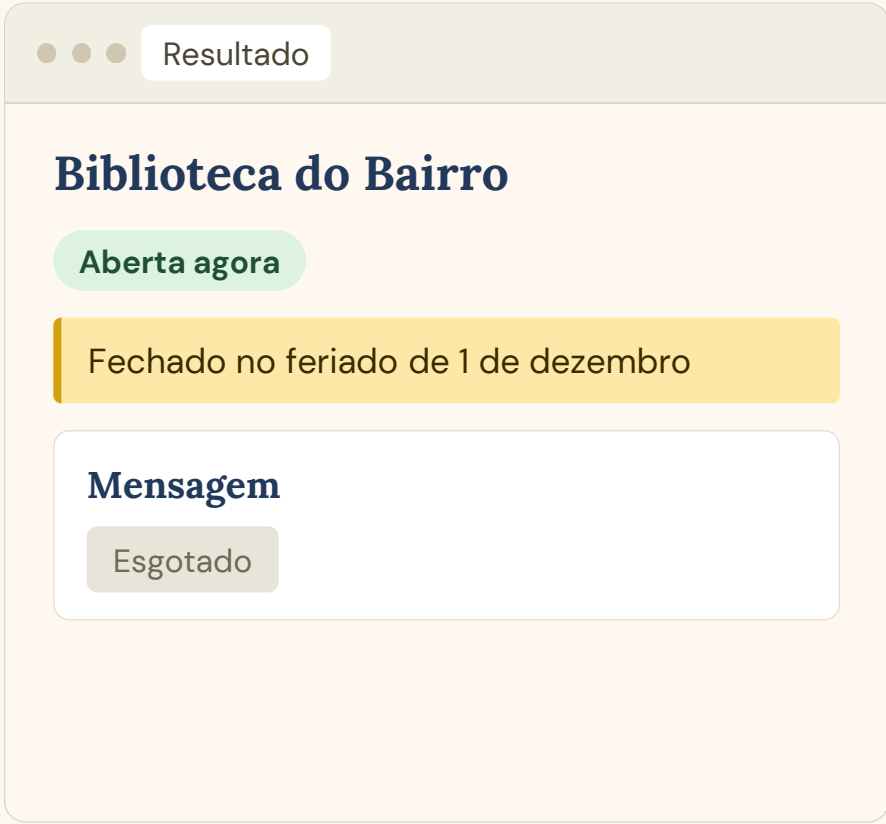
app.js

```
const estado = document.querySelector("#estado");
estado.textContent = "Aberta agora";

const aviso = document.querySelector(".aviso");
aviso.classList.add("destaque");

const botao = document.querySelector("#reservar");
botao.disabled = true;
botao.textContent = "Esgotado";
```

Resultado



Criar elementos a partir de dados

app.js

```
const lista = document.querySelector("#lista-livros");

for (const livro of catalogo) {
  const item = document.createElement("li");
  item.textContent = `${livro.titulo} (${livro.ano})`;
  if (!livro.disponivel) {
    item.classList.add("indisponivel");
  }
  lista.append(item);
}
```

Resultado

Catálogo

Os Maias (1888)

~~Mensagem (1934)~~ emprestado

A Sibila (1954)

EXERCÍCIO 07 · 30 MIN

O catálogo na página

01 No catalogo.html, deixe só um <ul id="lista-livros"> vazio

02 Com o array do exercício 6, crie um <li> por livro

03 Cada item mostra «Título — Autor (ano)»

04 Os emprestados levam a classe indisponivel e a palavra «emprestado»

05 No <h2>, escreva o resumo: «6 livros, 4 disponíveis»

Concluído quando…

- Mudar o array muda a página, sem tocar no HTML
- O resumo do <h2> é calculado, não escrito à mão
- Não usa innerHTML com os dados dos livros

Eventos: reagir ao utilizador

app.js

```
const botao = document.querySelector("#tema");

botao.addEventListener("click", () => {
  document.body.classList.toggle("escuro");
});

const pesquisa = document.querySelector("#pesquisa");

pesquisa.addEventListener("input", () => {
  console.log("Escreveu:", pesquisa.value);
});
```

Eventos mais usados

`click`	Clicar ou tocar
`input`	Cada tecla num campo
`change`	Escolha num select ou checkbox
`submit`	Enviar um formulário
`keydown`	Carregar numa tecla

Validar um formulário

app.js

```
const form = document.querySelector("#inscricao");
const erro = document.querySelector("#mensagem");

form.addEventListener("submit", (evento) => {
  evento.preventDefault();   // não recarregar a página
  const nome = form.nome.value.trim();
  const email = form.email.value.trim();
  if (nome === "" || !email.includes("@")) {
    erro.textContent = "Preencha o nome e um e-mail válido.";
    return;
  }
  erro.textContent = `Obrigado, ${nome}! Inscrição feita.`;
});
```

Resultado



EXERCÍCIO 08 · 40 MIN

Pesquisar e inscrever

01 Caixa de pesquisa no catálogo: a lista filtra a cada tecla

02 A pesquisa ignora maiúsculas: «maias» encontra «Os Maias»

03 Botão «Só disponíveis» que liga e desliga esse filtro

04 Formulário de inscrição validado sem recarregar a página

05 Mensagem de erro ou de sucesso junto ao formulário

Concluído quando…

- Apagar a pesquisa volta a mostrar todos os livros
- Sem resultados, aparece «Nenhum livro encontrado»
- Pesquisa e «Só disponíveis» funcionam em conjunto

Lembrar com localStorage

app.js

```
// Guardar e ler texto
localStorage.setItem("tema", "escuro");
const tema = localStorage.getItem("tema");
if (tema === "escuro") {
  document.body.classList.add("escuro");
}

// Arrays e objetos: converter com JSON
const favoritos = ["Os Maias", "A Sibila"];
localStorage.setItem("favoritos", JSON.stringify(favoritos));

const texto = localStorage.getItem("favoritos");
const lista = JSON.parse(texto) ?? [];
```

DevTools › Application

Key	Value
tema	escuro
favoritos	["Os Maias", "A Sibila"]

> Fica guardado mesmo depois de fechar o browser

> Só guarda texto: **JSON** converte listas e objetos

Carregar dados com fetch

app.js

```
async function carregarLivros() {
  try {
    const resposta = await fetch("livros.json");
    const livros = await resposta.json();
    mostrarLivros(livros);
  } catch (erro) {
    lista.textContent = "Não foi possível carregar o catálogo.";
  }
}

carregarLivros();
```

livros.json

```
[
  {
    "titulo": "Os Maias",
    "ano": 1888,
    "disponivel": true
  },
  …
]
```

await espera pela resposta sem bloquear a página

try / catch trata a falha

EXERCÍCIO 09 · 40 MIN

Lembrar e carregar

01 Passe o catálogo do app.js para um ficheiro livros.json

02 Carregue-o com fetch e mostre-o com mostrarLivros

03 Mostre uma mensagem na página se o ficheiro não carregar

04 Botão de modo escuro, lembrado com localStorage

05 Botão «Guardar» em cada livro; os favoritos ficam guardados

Concluído quando…

- Ao recarregar, o tema e os favoritos mantêm-se
- Acrescentar um livro ao JSON muda o catálogo
- Mudar o nome do ficheiro mostra a mensagem de erro

Quando alguma coisa falha

01 Abrir a consola e ler a mensagem toda

02 Clicar em app.js:12 para ir à linha

03 Espreitar valores com console.log

04 Pôr um breakpoint em Sources e avançar passo a passo

Consola

```
Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')
```
app.js:12

null = o querySelector não encontrou nada: seletor errado ou falta o defer

```
Uncaught ReferenceError: livro is not defined
```
app.js:20

Nome mal escrito, ou variável usada fora do seu bloco

Erros comuns e como resolver

O que aparece	Causa provável
x is not defined	Nome mal escrito, maiúsculas, ou fora do bloco
Cannot read properties of null	O querySelector não encontrou: seletor ou defer
… is not a function	Método mal escrito (console.lg) ou não existe
Assignment to constant variable	Tentou mudar uma const: usar let
Unexpected token	Chaveta, parêntese ou vírgula a mais ou a menos
"5" + 2 dá "52"	O valor do campo é texto: converter com Number()
A página recarrega ao enviar	Falta evento.preventDefault() no submit
Nada acontece, sem erros	O script não está ligado, ou o if usa = em vez de ===

Cábula de JavaScript

Valores

```
const nome = "…";
let total = 0;
typeof valor
`Olá, ${nome}`
Number("5")
```

Decisões

```
if (a === b) { }
else if (…) { }
else { }
&&  ||  !
>  <  >=  !==
```

Funções

```
function f(x) {
  return x * 2;
}
const g = (x) => x * 2;
f(10)
```

Arrays e objetos

```
lista.push(x)
lista.length
for (const x of lista)
map · filter · find
livro.titulo
```

DOM

```
querySelector("…")
el.textContent = "…"
el.classList.toggle()
createElement("li")
pai.append(filho)
```

Eventos e dados

```
addEventListener()
e.preventDefault()
localStorage.setItem()
JSON.parse · stringify
await fetch("…")
```

PROJETO FINAL · INDIVIDUAL OU A PARES · 3 AULAS

A Biblioteca do Bairro interativa

01 Parta do site com HTML e CSS dos cursos anteriores

02 Um app.js organizado em funções com nomes claros

03 Os dados vêm de ficheiros JSON, não do HTML

04 Tudo funciona com rato, teclado e no telemóvel

05 Publique no GitHub Pages e apresente à turma

O que cada página faz

`index`	«Aberta agora» com a hora real
`catalogo`	Pesquisa, filtro e favoritos
`eventos`	Próximos eventos, de um JSON
`contactos`	Formulário validado
`todas`	Modo escuro lembrado

PROJETO FINAL · ANTES DE ENTREGAR

Lista de verificação

- Nenhum erro vermelho na consola
- Código em funções com nomes claros
- const por omissão; let só quando o valor muda
- Comparações sempre com === e !==
- Clicáveis são <button> e funcionam com teclado
- Dados em livros.json e eventos.json
- Elementos criados sem innerHTML com dados
- Pesquisa e filtros funcionam em conjunto
- O formulário explica o que está mal
- Tema e favoritos lembrados depois de recarregar

E AGORA?

Próximos passos

Praticar um pouco por dia

JavaScript30 e freeCodeCamp têm projetos curtos

Consultar a MDN

A referência da linguagem e do DOM, com exemplos

APIs públicas

Dados reais com fetch: meteorologia, livros, mapas

Módulos e npm

import, export e bibliotecas de outras pessoas

Frameworks

React ou Vue, depois de dominar o JavaScript puro

O percurso completo

HTML, CSS, Git e JavaScript: sites completos, interativos e publicados

ANEXO

Soluções

return solucao;

Uma proposta de resolução para cada exercício. Em programação há muitas soluções certas: o importante é funcionar e ler-se bem.

SOLUÇÃO · EXERCÍCIO 01

Olá, consola

No <head>: **<script src="app.js" defer></script>**

A mensagem aparece em F12 › Console, não na página

O erro indica o ficheiro e a linha: app.js:9

app.js

```
// Biblioteca do Bairro — Ana Silva
console.log("Bem-vindos à Biblioteca do Bairro!");

// Na consola:
// > 2026 - 1987
// ← 39

// Erro de propósito (linha 9):
console.lg("teste");
// TypeError: console.lg is not a function
```

SOLUÇÃO · EXERCÍCIO 02

Os dados da biblioteca

- Nome e morada não mudam: **const**
- Os livros disponíveis mudam: **let**
- A frase usa a variável, não o número escrito

app.js

```
const nome = "Biblioteca do Bairro";
const morada = "Rua das Flores, 12";
let livros = 1250;

livros = livros - 3;      // três empréstimos
console.log(livros);       // 1247

console.log(`A ${nome} tem ${livros} livros disponíveis.`);
console.log(typeof nome, typeof morada, typeof livros);
console.log(nome.toUpperCase());
```

SOLUÇÃO · EXERCÍCIO 03

Aberta ou fechada?

- Primeiro o caso mais simples: o domingo
- **< 19** e não <= 19: às 19 h já fechou
- **condição ? a : b** é um if numa só linha

app.js

```
const dia = "sábado";
const hora = 11;
let aberta;

if (dia === "domingo") {
  aberta = false;
} else if (dia === "sábado") {
  aberta = hora >= 10 && hora < 13;
} else {
  aberta = hora >= 9 && hora < 19;
}

console.log(aberta ? "Aberta" : "Fechada");
```

SOLUÇÃO · EXERCÍCIO 04

Funções da biblioteca

Cada função faz uma só coisa e devolve o resultado

mensagemMulta reutiliza as outras duas

estaAberta: o código do exercício 3, com return

app.js

```
function calcularMulta(dias) {
  return dias > 15 ? (dias - 15) * 0.5 : 0;
}
function formatarEuros(valor) {
  return valor.toFixed(2).replace(".", ",") + " €";
}
function mensagemMulta(dias) {
  const multa = calcularMulta(dias);
  if (multa === 0) return "Sem multa";
  return `Multa a pagar: ${formatarEuros(multa)}`;
}

console.log(mensagemMulta(10));   // Sem multa
console.log(mensagemMulta(22));   // Multa a pagar: 3,50 €
```

SOLUÇÃO · EXERCÍCIO 05

Lista de leituras

- **push** aceita vários valores de uma vez
- **forEach** também dá a posição: (titulo, i)
- **filter** e **map** criam arrays novos

app.js

```
const livros = ["Os Maias", "Mensagem", "A Sibila",
  "Os Lusíadas", "Aparição"];
livros.push("Memorial do Convento", "Ensaio sobre a Cegueira");
console.log(livros.length);   // 7

livros.forEach((titulo, i) => {
  console.log(`${i + 1}. ${titulo}`);
});

const longos = livros.filter(t => t.length > 10);
const maiusculas = livros.map(t => t.toUpperCase());
console.log(longos, maiusculas);
```

SOLUÇÃO · EXERCÍCIO 06

O catálogo em objetos

> O array catalogo tem seis objetos, como no slide 22

> Contar = filtrar e ver o **length**

> Em emprestar, um return termina cada caso

app.js

```
for (const l of catalogo) {
  console.log(`${l.titulo} — ${l.autor} (${l.ano})`);
}
console.log(catalogo.filter(l => l.disponivel).length);
let antigo = catalogo[0];
for (const l of catalogo) {
  if (l.ano < antigo.ano) antigo = l;
}
function emprestar(titulo) {
  const livro = catalogo.find(l => l.titulo === titulo);
  if (!livro) return "Livro não encontrado.";
  if (!livro.disponivel) return "Já está emprestado.";
  livro.disponivel = false; return "Emprestado!";
}
```

SOLUÇÃO · EXERCÍCIO 07

O catálogo na página

lista e **resumo**: o ul e o h2, com querySelector

replaceChildren() limpa antes de desenhar

No fim: **mostrarLivros(catalogo);**

app.js

```
function mostrarLivros(livros) {
  lista.replaceChildren();
  for (const l of livros) {
    const item = document.createElement("li");
    item.textContent = `${l.titulo} — ${l.autor} (${l.ano})`;
    if (!l.disponivel) {
      item.classList.add("indisponivel");
      item.append(" · emprestado");
    }
    lista.append(item);
  }
  const n = livros.filter(l => l.disponivel).length;
  resumo.textContent = `${livros.length} livros, ${n} disponíveis`;
}
```

SOLUÇÃO · EXERCÍCIO 08

Pesquisar e inscrever

- Os dois eventos chamam a mesma função
- Comparar tudo em minúsculas
- Formulário: o código do slide 29

app.js

```
let soDisponiveis = false;
function atualizar() {
  const texto = pesquisa.value.trim().toLowerCase();
  let res = catalogo.filter(l =>
    l.titulo.toLowerCase().includes(texto));
  if (soDisponiveis) res = res.filter(l => l.disponivel);
  mostrarLivros(res);
  if (!res.length) lista.textContent = "Nenhum livro encontrado.";
}
pesquisa.addEventListener("input", atualizar);
botao.addEventListener("click", () => {
  soDisponiveis = !soDisponiveis;
  atualizar();
});
```

SOLUÇÃO · EXERCÍCIO 09

Lembrar e carregar

catalogo passa a **let catalogo = [];** e enche-se com o JSON

toggle devolve true se a classe ficou ligada

Favoritos: um array guardado com JSON.stringify

```
async function carregarLivros() {
  try {
    const resposta = await fetch("livros.json");
    catalogo = await resposta.json();
    atualizar();
  } catch {
    lista.textContent = "Erro ao carregar o catálogo.";
  }
}
carregarLivros();

const tema = localStorage.getItem("tema");
document.body.classList.toggle("escuro", tema === "escuro");
botaoTema.addEventListener("click", () => {
  const escuro = document.body.classList.toggle("escuro");
  localStorage.setItem("tema", escuro ? "escuro" : "claro");
});
```